

RELATÓRIO DE AUDITORIA TÉCNICA, FORENSE E DE CONFORMIDADE

Avaliação de Segurança da Informação, LGPD, Stack Tecnológica e Arquitetura no projeto **GSencrypt**

Autor/Auditor: Prof. Dr. Fabiano Menegidio (Universidade de Mogi das Cruzes - UMC)

Data da Avaliação: Junho de 2026

Resumo

O presente documento estabelece um parecer técnico de auditoria forense baseado nos controles da ABNT NBR ISO/IEC 27001:2022 e na Lei Geral de Proteção de Dados (Lei nº 13.709/2018). O escopo da avaliação abrange a resiliência arquitetural, a stack tecnológica e as práticas de Engenharia de Software aplicadas no ecossistema **GSencrypt**, um gerenciador de credenciais de arquitetura de Conhecimento Zero (*Zero-Knowledge*). A análise contempla verificações de integridade de código, gestão de riscos operacionais e adequação ao *Privacy by Design*.

1 Dados de Identificação, Stack Tecnológica e Contexto Arquitetural

1.1 Identificação do Projeto

O **GSencrypt** consiste em um cofre digital projetado para mitigar vetores de ataque através da descentralização da custódia de chaves. O repositório contém módulos críticos de identidade (*accounts, authentication*), proteção e auditoria (*vault, audit*), conformidade (*lgpd*) e recuperação (*recovery*).

1.2 Mapeamento Detalhado da Stack Tecnológica

- **Linguagem Base:** Python 3.12 (Ambiente *slim* containerizado).
- **Framework Web:** Django 6.0.4.
- **Persistência e Banco de Dados:** PostgreSQL 15 integrado via *psycopg2-binary* e *dj-database-url*.
- **Criptografia e Hashing:** *cryptography* 42.0.0 (Implementação de AES-256 via Fernet), *argon2-cffi* 25.1.0 para derivação de chaves.
- **Controle de Acesso e Segurança:** *django-axes* 8.3.1 (Prevenção de Força Bruta), *pyotp* (MFA/TOTP).
- **Infraestrutura/Deployment:** Docker e Docker Compose, utilizando isolamento em *internal_network*.

1.3 Análise de Arquitetura de Software

O sistema adota o padrão **Monolito Modularizado**. Apesar de rodar em um único processo principal, as responsabilidades de negócio são fortemente isoladas em fronteiras lógicas (*boundaries*) representadas pelos "apps" do Django. O *Data Flow* garante que a camada de visão não

manipule senhas em texto claro; as interações ocorrem através de instâncias de Fernet instanciadas em tempo de execução no *Model CredencialCofre*. Existe um ponto crítico de infraestrutura detectado: o uso do comando `python manage.py runserver` no `docker-compose.yml`. Este é um servidor síncrono e não apto para produção, gerando gargalos severos de I/O e quebrando premissas de disponibilidade.

1.4 Enquadramento Regulatório pela LGPD

A natureza dos dados armazenados (Nomes, E-mails e Credenciais/Senhas) os qualifica como **Dados Pessoais (Art. 5º, I)**. Embora credenciais financeiras e de acesso não sejam elencadas estritamente como "Dados Sensíveis" sob o Art. 5º, II da LGPD (que se restringe a origem racial, religião, saúde, biometria, etc.), o *impacto financeiro e moral* de um vazamento exige tratamento de altíssimo rigor, equiparando as salvaguardas técnicas às de dados sensíveis. O sistema atende brilhantemente aos direitos do titular (Art. 18), possuindo portabilidade e descarte via rotas `/lgpd/exportar` e `/lgpd/excluir`.

2 Avaliação de AI-Assisted Development & Metodologia (Vibecoding)

2.1 Classificação da Metodologia

O repositório apresenta características de desenvolvimento **Híbrido com forte presença de AI-Driven Development (Vibecoding)**. Evidencia-se isso pela presença de comentários literais mapeando exigências acadêmicas (ex: `# Item 1.10 Logout da sessão atual` ou `# Log de Sucesso no Login item 5.1`), sugerindo que *prompts* direcionados ao LLM foram utilizados para gerar trechos de código baseados em um edital ou checklist.

2.2 Evidências de Qualidade e Manutenibilidade

1. **Alto Acoplamento em Hardcodes:** No arquivo `recovery/views.py`, a URL de redefinição (`http://127.0.0.1:8000/recovery/reset/`) está *hardcoded*. Isso viola os princípios DRY e prejudica a implantação em diferentes ambientes (Staging/Production).
2. **Tratamento Genérico de Exceções:** Em diversas *Views* (`accounts/views.py`, `authentication/views.py`), observa-se o bloco `except Exception as e:`, que mascara falhas específicas (como `DatabaseError` ou `ValidationError`) com respostas 500 genéricas, dificultando a observabilidade arquitetural.
3. **Segurança WORM Excelente:** A implementação de logs imutáveis em `audit/models.py` subrescrevendo `save()` e `delete()` para lançar `PermissionDenied` é um padrão arquitetural de segurança defensiva brilhante e maduro.

Code Quality / AI Usage Score: 82/100. O débito técnico concentra-se na falta de adequação para *deploy* real (falta `gunicorn/nginx`, `hardcodes`) e no uso excessivo de blocos genéricos de exceção, porém a lógica de negócios criptográfica e de LGPD foi excepcionalmente bem orquestrada pelo LLM/Desenvolvedor.

3 Diagnóstico Baseado na Tríade SIA/CIDA & Scores

- **Confidencialidade (Score: 95/100):** Uso impecável de AES-256 (Fernet) acoplado à `SECRET_KEY` do ambiente para proteção de dados em repouso.

- **Integridade (Score: 88/100):** O banco de dados PostgreSQL e os modelos garantem tipagem estrita. A tabela de auditoria (AuditLog) atua como WORM (*Write Once, Read Many*), garantindo trilha de auditoria inviolável pela própria aplicação.
- **Disponibilidade (Score: 65/100):** O gargalo principal da arquitetura. O runserver utilizado no Docker é *single-threaded* e sucumbirá facilmente a sobrecargas. Faltam delimitadores de recursos de CPU/RAM.
- **Autenticidade (Score: 92/100):** Controle de acesso refinado com verificação inicial separada do desafio de 2FA (TOTP e E-mail), mitigando vetores de ataque em credenciais vazadas.

4 Modelagem de Ameaças e Resiliência a Ataques

A análise da superfície de ataque do **GSencript** detectou:

- **Vulnerabilidades de Injeção:** Devido à utilização do ORM do Django, a aplicação está majoritariamente imune a *SQL Injection* de primeira ordem.
- **Insecure Direct Object Reference (IDOR):** A arquitetura demonstrou estar blindada contra IDOR nas rotas do cofre. As consultas nas *views* de exclusão e edição realizam restrição mandatória: `CredencialCofre.objects.get(id=senha_id, usuario=request.user)`.
- **Supply Chain Attacks:** O projeto fixou agressivamente suas dependências (Django==6.0.4, cryptography==42.0.0) no `requirements.txt`, mitigando a inclusão arbitrária de malwares em pacotes atualizados, o que denota alta maturidade.
- **Resource Exhaustion (Denial of Service):** A ausência de limites configurados na ingestão de requisições HTTP (por exemplo, tamanho máximo de *payloads* JSON ou requisições simultâneas no WSGI) permite ataques de assimetria de recursos.

5 Auditoria Completa Baseada em Checklist de Requisitos

Tabela 1: Checklist de Validação Arquitetural e de Segurança

ID	Requisito	Status	Justificativa Técnica		
1.1	Hash criptográfico seguro	[CONF.]	Implementado	Argon2id	via CustomArgon2Hasher em settings.py.
1.2	Parâmetros de custo justificados	[CONF.]	Tempo = 2, Memória = 65536 KB, Paralelismo = 1 (Aderente OWASP).		
1.3	Uso de salt criptográfico único	[CONF.]	Gerenciado nativamente pela arquitetura de hash do Django.		
1.4	Armazenamento correto	[CONF.]	Persistido em campo do AbstractUser customizado.		
1.5	Autenticação 2FA	[CONF.]	App e E-mail 2FA (pyotp e Token DB) presentes em authentication.		

Continua na próxima página...

Tabela 1 – continuação da página anterior

ID	Requisito	Status	Justificativa Técnica
1.6	Validação após primária	[CONF.]	Fluxo exige checagem de senha e só depois verifica o TOTP em <code>views.py</code> .
1.7	Fluxo documentado	[CONF.]	Presente no <code>README.md</code> de forma explícita.
1.8	Evidências funcionais	[CONF.]	Módulo exibe funcionamento lógico nos testes em <code>views</code> .
1.9	Sessões com expiração	[CONF.]	<code>SESSION_COOKIE_AGE</code> = 1800 (30 minutos) configurado.
1.10	Invalidação no logout	[CONF.]	Rota de logout destrói os dados de sessão no servidor.
1.11	Proteção contra força bruta	[CONF.]	Pacote <code>django-axes</code> configurado para bloquear após 5 falhas.
1.12	Justificativas técnicas	[CONF.]	Arquivo PDF e DOCX detalham o racional técnico.
2.1	Recuperação de senha implementada	[CONF.]	Implementado em <code>recovery/views.py</code> .
2.2	Token seguro	[CONF.]	Uso de <code>secrets.token_urlsafe(32)</code> . Alta entropia.
2.3	Token com expiração	[CONF.]	Definida validade de 15 minutos em <code>token_expiration</code> .
2.4	Token invalidado após uso	[CONF.]	Wiping efetuado explicitamente após validação.
2.5	Falha correta para expirado	[CONF.]	Wording de erro apropriado sem vaziar vetores de exploração.
2.6	Log de solicitação	[CONF.]	<code>AuditLog</code> registra <code>RECUPERACAO_SOLICITADA</code> .
2.7	Log de sucesso/falha	[CONF.]	Registros atrelados via <code>*Signals*</code> e chamadas manuais.
3.1	Comunicação HTTPS/TLS	[N.C.]	<code>SECURE_SSL_REDIRECT</code> = <code>False</code> no <code>settings.py</code> (aceitável apenas em dev).
3.2	Bloqueio de conexão insegura	[N.C.]	HSTS configurado com <code>True</code> , mas sem o redirecionamento SSL efetivo em infra.
3.3	Tráfego cifrado	[N.E.]	Falta proxy reverso (Nginx/Traefik) no Compose.
3.4	Cripto em repouso	[CONF.]	Implementado para as senhas salvas (<code>senha_site_cifrada</code>).
3.5	Algoritmo criptográfico adequado	[CONF.]	Fernet provê AES em modo CBC/GCM com chaves seguras de 256 bits.
3.6	Chaves protegidas	[CONF.]	<code>SECRET_KEY</code> lida por variável de ambiente (<code>.env</code>).
3.7	Estratégia documentada	[CONF.]	Explicitado no documento Word e Github.
3.8	Justificativa das escolhas	[CONF.]	Atende OWASP Top 10 para proteção de dados sensíveis.

Continua na próxima página...

Tabela 1 – continuação da página anterior

ID	Requisito	Status	Justificativa Técnica
4.1	Lista de dados coletados	[CONF.]	Explícito no T&C e <i>Privacy Dashboard</i> .
4.2	Associação a finalidade	[CONF.]	ConsentimentoLGPD atrela aceite a finalidade específica.
4.3	Minimização de dados	[CONF.]	Solicita apenas e-mail, usuário e nome na camada primária.
4.4	Consentimento explícito	[CONF.]	Flag booleana <code>aceite_termos</code> no banco.
4.5	Consentimento associado	[CONF.]	Registrado na tabela relacional ligada ao <code>User</code> .
4.6	Revogação	[CONF.]	Rota <code>/lgpd/revogar/</code> e método ativo na model.
4.7	Registro de versão	[CONF.]	Armazena data, hora exata e a <i>string</i> <code>versao_termo</code> .
4.8	Consulta aos dados	[CONF.]	<code>/lgpd/consultar/</code> exhibe JSON detalhado ao titular.
4.9	Exportação	[CONF.]	Resposta forçada como <code>attachment</code> para portabilidade.
4.10	Exclusão dos dados	[CONF.]	Cascata garantida pelo ORM do Django. Exclusão irreversível.
4.11	Fluxo documentado	[CONF.]	Mapeado no DOCX.
5.1	Logs de autenticação	[CONF.]	Utilização elegante do <i>django-signals</i> .
5.2	Logs de falhas	[CONF.]	Integração do <i>django-axes</i> enviando <i>signals</i> para <code>AuditLog</code> .
5.3	Proteção contra alteração	[CONF.]	Camada <code>save()</code> restringe a sobrescrita de IDs de auditoria (WORM).
5.4	Exemplo de análise	[CONF.]	Dashboard administrativo fornece visualização formatada.

6 Matriz de Risco, Plano de Ação e Remediação

Tabela 2: Matriz de Riscos de Stack e Arquitetura

Vulnerabilidade	Pilar	Nível	Impacto Técnico	Plano de Ação
1. Servidor de Desenvolvimento em Produção	D	Crítica	Gargalos síncronos e <i>crashes</i> inesperados sob carga média.	Trocar o comando <code>runserver</code> por <code>gunicorn</code> no <code>Dockerfile</code> .

Continua na próxima página...

Tabela 2 – continuação da página anterior

Vulnerabilidade	Pilar	Nível	Impacto Técnico	Plano de Ação
2. Hardcode de URLs de Redirecionamento	C, I	Média	Geração de links quebrados ou vetores de <i>Open Redirect</i> em infraestruturas distribuídas.	Utilizar <code>request.build_absolute_uri()</code> para montagem dinâmica.
3. Inexistência de Proteção de Carga Útil (<i>Payload Limit</i>)	D, I	Alta	Atacante pode esgotar memória RAM enviando JSON infinito (<i>Billion Laughs / Large Blob</i>).	Implementar um <i>Middleware</i> customizado para travar grandes requests.
4. Tráfego não Criptografado	C	Crítica	Redirecionamento de tráfego HTTP desprotegido em ambientes Cloud.	Configurar <code>SECURE_SSL_REDIRECT = True</code> nos <i>settings</i> .

6.1 Trechos de Código Corretivo e Arquitetura Segura

Abaixo, apresentamos os *snippets* fundamentais (Clean Code e Secure Coding) em Python para mitigar as vulnerabilidades arquiteturais detectadas.

6.1.1 1. Substituição Responsiva do URL Hardcoded (Redução de Acoplamento)

No arquivo `recovery/views.py`, para mitigar *URL Spoofing* e adequar a arquitetura a diferentes *environments* dinamicamente:

```

1 # Arquivo: recovery/views.py
2 from django.urls import reverse
3
4 # Na view solicitar_recuperacao:
5 # Em vez de url_redefinicao = f"http://127.0.0.1:8000/recovery/reset/..."
6 path = reverse('resetar_senha')
7 url_redefinicao = request.build_absolute_uri(f"{path}?token={user.recovery_token}")

```

Listing 1: Geração dinâmica e segura de links absolutos

6.1.2 2. Middleware para Proteção de Esgotamento de Recursos (Guardrail de Memória)

Arquiteturas monolíticas estão suscetíveis ao estouro de limite de requisição. O código abaixo cria um *Middleware* limitador (impede *Resource Exhaustion DoS*):

```

1 # Arquivo: core/middleware/security_guard.py
2 from django.http import HttpResponseBadRequest
3
4 class PayloadSizeLimitMiddleware:

```

```
5 def __init__(self, get_response):
6     self.get_response = get_response
7     self.MAX_CONTENT_LENGTH = 1048576 # 1 MegaByte
8
9 def __call__(self, request):
10     content_length = request.META.get('CONTENT_LENGTH')
11     if content_length and int(content_length) > self.MAX_CONTENT_LENGTH:
12         return HttpResponseBadRequest("Tamanho maximo da requisicao excedido
13     .")
14
15     return self.get_response(request)
```

Listing 2: Implementação de Guardrail para Payloads HTTPS

6.1.3 3. Arquivo de Execução Robusto para o Dockerfile

O runserver deve ser completamente erradicado do fluxo de *deploy* containerizado. É imperativo adotar um servidor WSGI real e configurado para limites operacionais seguros.

```
1 # Instale no requirements.txt: gunicorn==21.2.0
2 # Modifique o comando no final do Dockerfile e/ou docker-compose.yml:
3
4 CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "3", "--limit-request-
    line", "4094", "core.wsgi:application"]
```

Listing 3: Atualização de segurança para o entrypoint Docker

7 Conclusão do Parecer de Auditoria & Referências Bibliográficas

Com base na auditoria exaustiva do código-fonte e na modelagem arquitetural submetida, **homologo o projeto GSencrpt** com as ressalvas pontuadas no plano de ação. A aplicação demonstra um elevado grau de maturidade no tange à conformidade com a LGPD (Art. 18 - autodeterminação informativa) e aderência ao padrão OWASP (implementação correta de algoritmos criptográficos AES e Argon2, mitigação de senhas expostas, trilhas de log imutáveis). O esforço focado em *Zero-Knowledge Architecture* atende estritamente às premissas de *Privacy by Design*. Para alocação em ambiente de produção, é **mandatória** a substituição da engine de carregamento de páginas (*gunicorn/nginx*) e a implantação de um certificado TLS autêntico.

Referências Bibliográficas

- ABNT NBR ISO/IEC 27001:2022 - *Tecnologia da informação - Técnicas de segurança - Sistemas de gestão da segurança da informação - Requisitos.*
- LEI Nº 13.709, DE 14 DE AGOSTO DE 2018 - *Lei Geral de Proteção de Dados Pessoais (LGPD).*
- OWASP. (2021). *OWASP Top 10:2021 - Identificação e Mitigação de Riscos em Aplicações Web.*
- MARTIN, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall, 2017.